

Version : 1.0.0 Dernière mise à jour : 17 avril 2026 Auteurs : Équipe MatchPet Licence : Privé

## Table des matières

- 1. [Introduction](#)
- 2. [Stack technologique](#)
- 3. [Architecture](#)
- 4. [Arborescence du projet](#)
- 5. [Modèle de données](#)
- 6. [Référence API](#)
- 7. [Logique métier — Moteur de matching](#)
- 8. [Expérience utilisateur](#)
- 9. [Installation & Développement](#)
- 10. [Déploiement & CI/CD](#)
- 11. [Sécurité & Bonnes pratiques](#)
- 12. [Maintenance](#)
- 13. [Améliorations futures](#)

## 1. Introduction

**MatchPet** est une application web **mobile-first** dédiée à la mise en relation intelligente entre des adoptants et des animaux issus de refuges.

Le système repose sur un pilier principal :

**Compatibilité de mode de vie** via un moteur de matching basé sur un questionnaire structuré.

Le projet est actuellement déployé à l'échelle du **département de la Seine-et-Marne (77)**, qui constitue son périmètre d'utilisation.

### Objectifs de cette documentation

| Objectif       | Description                                                            |
|----------------|------------------------------------------------------------------------|
| Maintenabilité | Permettre à tout développeur de comprendre et modifier le projet       |
| Pérennité      | Garantir la continuité technique en cas de changement d'équipe         |
| Référence      | Servir de source de vérité pour l'architecture et les choix techniques |

## 2. Stack technologique

### Frontend

| Technologie           | Version         | Rôle                                                                       |
|-----------------------|-----------------|----------------------------------------------------------------------------|
| Next.js               | 16.2.1          | Framework React — App Router, SSR/ISR, optimisation SEO                    |
| React                 | 19.2.4          | Bibliothèque JavaScript déclarative pour la création d'UI                  |
| TypeScript            | ^5              | Typage statique afin d'éviter les erreurs et assurer la robustesse du code |
| Tailwind CSS          | ^4              | Framework CSS                                                              |
| Framer Motion         | ^12.38.0        | Animations et transitions                                                  |
| React Three Fiber     | ^9.5.0          | Rendu 3D dans React (Three.js)                                             |
| Drei                  | ^10.7.7         | Helpers et abstractions pour R3F                                           |
| Lucide React          | ^0.577.0        | Bibliothèque d'icônes                                                      |
| clsx + tailwind-merge | ^2.1.1 / ^3.5.0 | Utilitaires de classNames conditionnels                                    |

## Backend

| Technologie            | Version | Rôle                                                   |
|------------------------|---------|--------------------------------------------------------|
| Node.js                | 22.22.2 | Runtime serveur (environnement d'exécution JavaScript) |
| Next.js API Routes     | 16.2.1  | Endpoints REST côté serveur (App Router)               |
| Prisma                 | ^7.5.0  | ORM — accès type-safe à la base de données             |
| Prisma MariaDB Adapter | ^7.7.0  | Driver adapter natif pour MariaDB                      |
| bcryptjs               | ^3.0.3  | Hachage des mots de passe                              |

## Base de données

| Technologie | Rôle                                     |
|-------------|------------------------------------------|
| MySQL       | Base de données relationnelle principale |

## Tests & Qualité

| Outil              | Version | Rôle                              |
|--------------------|---------|-----------------------------------|
| Jest               | ^29.7.0 | Framework de tests unitaires      |
| Testing Library    | ^16.2.0 | Tests de composants React         |
| ESLint             | ^9      | Linting et standards de code      |
| eslint-config-next | 16.2.1  | Règles ESLint spécifiques Next.js |

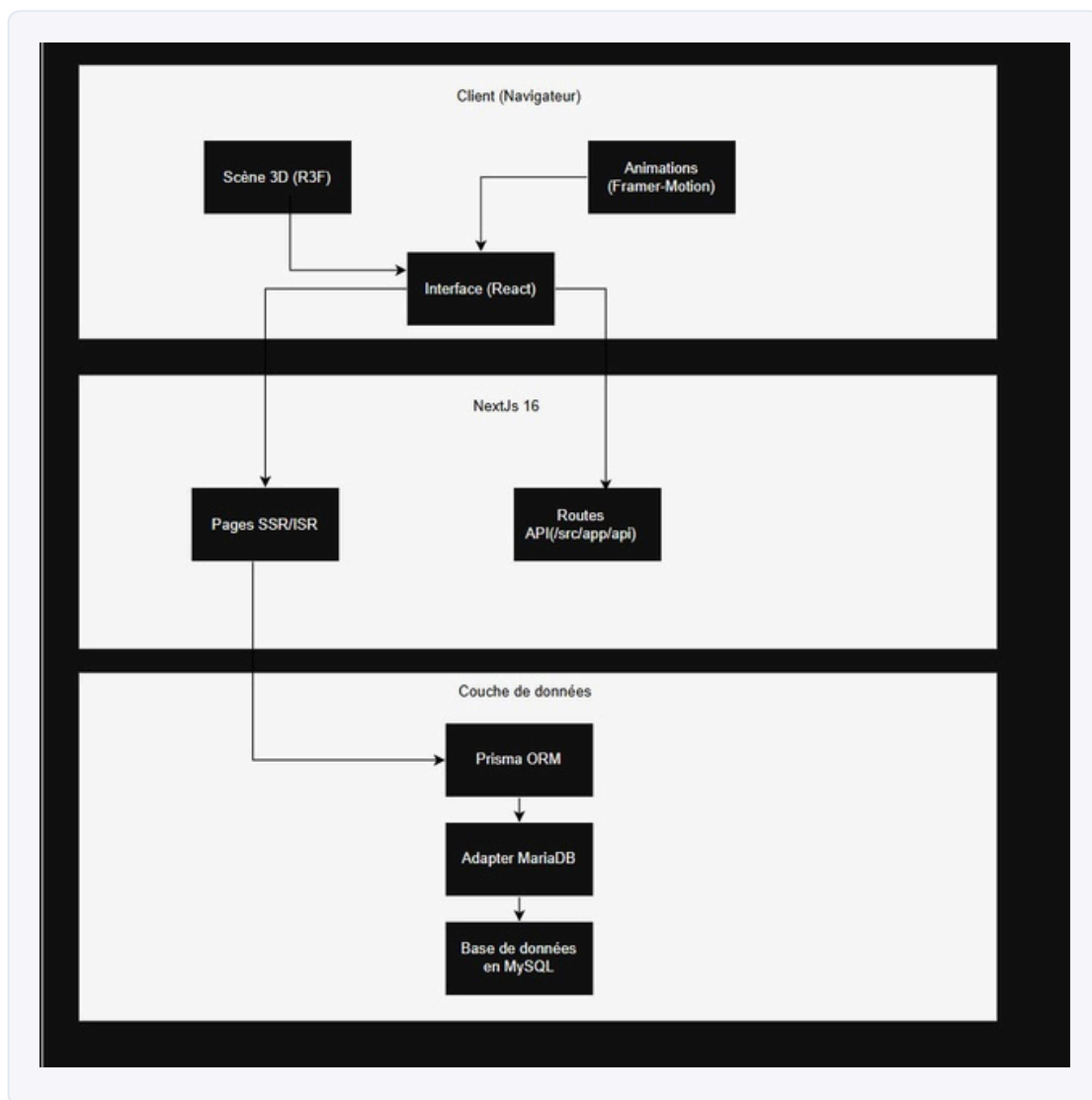
## Infrastructure

| Service        | Rôle                                                          |
|----------------|---------------------------------------------------------------|
| Railway        | Hébergement de l'application et de la base de données managée |
| GitHub Actions | Pipeline CI/CD (lint, build, tests)                           |

## 3. Architecture

L'application repose sur une **architecture full-stack monolithique** basée sur Next.js avec le pattern **App Router**.

## Diagramme d'architecture



### Couches applicatives

| Couche            | Responsabilité                             | Répertoire                                            |
|-------------------|--------------------------------------------|-------------------------------------------------------|
| Présentation      | Interface utilisateur, animations, 3D      | <code>src/app/</code> , <code>src/components/</code>  |
| Logique métier    | Endpoints REST, matching, authentification | <code>src/app/api/</code>                             |
| Accès aux données | Requêtes ORM, modèles, migrations          | <code>src/lib/prisma.ts</code> , <code>prisma/</code> |
| Assets            | Images, modèles 3D, SVG                    | <code>public/</code>                                  |

### Gestion des API sans Express

Le projet n'utilise **pas Express.js**. Les endpoints REST sont gérés nativement par le système d'**API Routes de Next.js (App Router)**, qui remplace le besoin d'un framework HTTP externe.

#### Principe de fonctionnement :

Chaque endpoint correspond à un fichier `route.ts` placé dans l'arborescence `src/app/api/`.

Le chemin du fichier détermine l'URL de l'endpoint :

```
src/app/api/match/route.ts      → GET    /api/match
src/app/api/login/route.ts      → POST  /api/login
src/app/api/profile/route.ts    → GET    /api/profile
                                PUT    /api/profile
src/app/api/admin/animals/route.ts → GET/POST /api/admin/animals
```

Chaque fichier exporte des **fonctions nommées** correspondant aux méthodes HTTP :

```
// src/app/api/match/route.ts
export async function GET(request: Request) {
  // Logique de traitement
  return NextResponse.json(data)
}

export async function POST(request: Request) {
  const body = await request.json()
  // ...
  return NextResponse.json({ success: true })
}
```

Comparaison avec Express :

| Aspect        | Express.js                                                      | Next.js API Routes (App Router)                                |
|---------------|-----------------------------------------------------------------|----------------------------------------------------------------|
| Routage       | Déclaratif via <code>app.get()</code> , <code>app.post()</code> | Basé sur le système de fichiers                                |
| Requête       | Objet <code>req</code> propriétaire Express                     | Objet <code>Request</code> standard (Web API)                  |
| Réponse       | Objet <code>res</code> propriétaire Express                     | <code>NextResponse</code> (extends Web <code>Response</code> ) |
| Middleware    | <code>app.use()</code> chaîné                                   | <code>middleware.ts</code> à la racine ou logique inline       |
| Serveur       | Serveur Express dédié                                           | Intégré au serveur Next.js                                     |
| Configuration | Manuelle (port, CORS, parsers)                                  | Zéro configuration                                             |

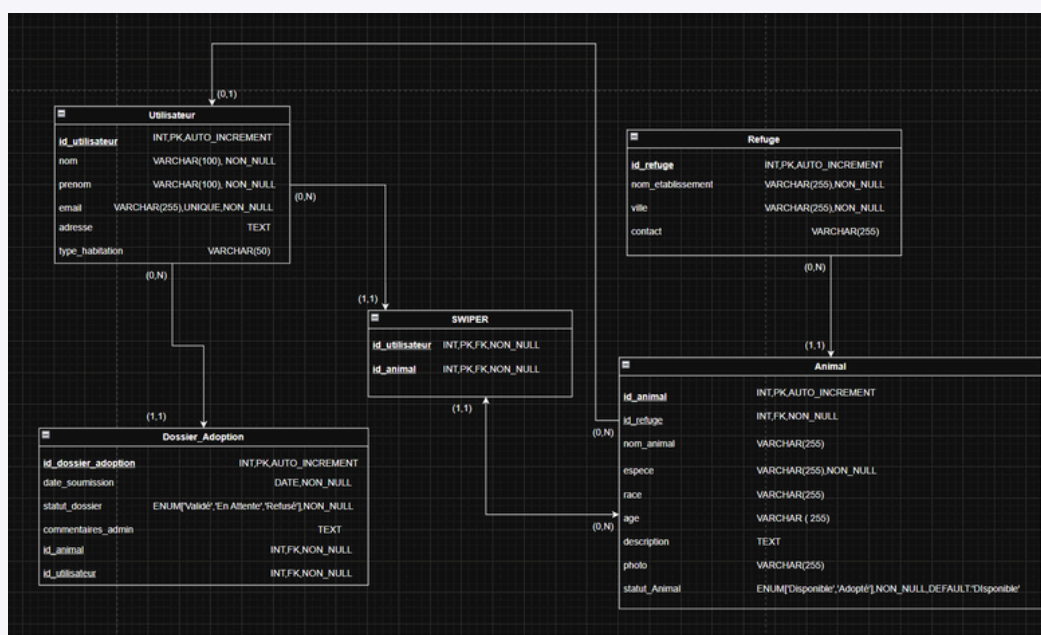
Le fichier `server.js` à la racine du projet est un **custom server** minimaliste qui utilise le module natif `http` de Node.js uniquement pour gérer le port dynamique sur Railway. Il délègue 100% du traitement des requêtes au handler Next.js.

4. Arborescence du projet

```
matchpet/      |      .github/      |      workflows/
|              |      |              |      |
|      └─ ci.yml      |      # Pipeline CI GitHub Actions
|              |      |              |      |
|      └─ migrations/      |      # Historique des migrations SQL
|              |      |      └─ 20260413183413_initial_schema/
|              |      |      # Schéma de la base de données
|              |      └─ schema.prisma
|              └─ seed.ts      # Script de données initiales
|              |      public/
|              └─ shiba_dog.glb      # Modèle 3D - Shiba (16 Mo)
|              └─ an_animated_cat.glb      # Modèle 3D - Chat animé
|              └─ rabbit.glb      # Modèle 3D - Lapin |      └─ logo-
avectitre.png      # Logo avec titre
|      └─ logo_sansntitre.png      # Logo sans titre
|      └─ img_nomatch.png      # Illustration "aucun résultat"
|      └─ src/      |      └─ app/
|      |      └─ layout.tsx      # Layout racine
|      |      └─ page.tsx      # Page d'accueil
|      |      └─ globals.css      # Styles globaux
|      |      └─ onboarding/      # Questionnaire d'onboarding
|      |      └─ match/      # Interface de matching (swipe)
|      |      └─ search/      # Recherche d'animaux
|      |      └─ adopt/      # Formulaire d'adoption
|      |      └─ adoptions/      # Suivi des adoptions (utilisateur)
|      |      └─ profile/      # Gestion du profil
|      |      └─ favorites/      # Favoris
|      |      └─ associations/      # Espace associations
|      |      |      └─ api/
|      |      |      └─ register/      # POST - Inscription utilisateur
|      |      |      └─ login/      # POST - Connexion utilisateur
|      |      |      └─ profile/      # GET/PUT - Profil utilisateur
|      |      |      └─ match/      # GET - Moteur de matching
|      |      |      └─ search/      # GET - Recherche libre
|      |      |      └─ adopt/      # POST - Demande d'adoption
|      |      |      └─ adoptions/      # GET - Liste des adoptions
|      |      |      └─ messages/      # GET/POST - Messagerie
|      |      |      |      └─ admin/
|      |      |      |      └─ register/      # POST - Inscription refuge
```

## 5. Modèle de données

### Diagramme Entité-Relation



Détail des entités

Utilisateur — Adoptant

Stocke les informations personnelles d'un utilisateur et son type d'habitation.

| Champ           | Type         | Contraintes        | Description                               |
|-----------------|--------------|--------------------|-------------------------------------------|
| id_utilisateur  | INT          | PK, AUTO_INCREMENT | Identifiant unique de l'utilisateur       |
| nom             | VARCHAR(100) | NON_NULL           | Nom de famille                            |
| prenom          | VARCHAR(100) | NON_NULL           | Prénom                                    |
| email           | VARCHAR(255) | UNIQUE, NON_NULL   | Adresse email de connexion                |
| adresse         | TEXT         | —                  | Adresse postale                           |
| type_habitation | VARCHAR(50)  | —                  | Type de logement (maison, appartement...) |

Refuge — Association / Refuge

Représente un refuge partenaire avec ses informations de contact.

| Champ             | Type         | Contraintes        | Description                  |
|-------------------|--------------|--------------------|------------------------------|
| id_refuge         | INT          | PK, AUTO_INCREMENT | Identifiant unique du refuge |
| nom_etablissement | VARCHAR(255) | NON_NULL           | Nom officiel du refuge       |
| ville             | VARCHAR(255) | NON_NULL           | Ville d'implantation         |
| contact           | VARCHAR(255) | —                  | Informations de contact      |

Animal — Animal à adopter

| Champ         | Type                         | Contraintes                   | Description                          |
|---------------|------------------------------|-------------------------------|--------------------------------------|
| id_animal     | INT                          | PK, AUTO_INCREMENT            | Identifiant unique de l'animal       |
| id_refuge     | INT                          | FK, NON_NULL                  | Refuge hébergeant l'animal           |
| nom_animal    | VARCHAR(255)                 | —                             | Nom de l'animal                      |
| espece        | VARCHAR(255)                 | NON_NULL                      | Espèce (chien, chat...)              |
| race          | VARCHAR(255)                 | —                             | Race de l'animal                     |
| age           | VARCHAR(255)                 | NON_NULL                      | Âge                                  |
| description   | TEXT                         | —                             | Description libre de l'animal        |
| photo         | VARCHAR(255)                 | —                             | URL de la photo                      |
| statut_Animal | ENUM('Disponible', 'Adopté') | NON_NULL, DEFAULT:'Disponible | Statut de disponibilité à l'adoption |

Dossier\_Adoption — Dossier d'adoption

| Champ               | Type                                   | Contraintes        | Description                                |
|---------------------|----------------------------------------|--------------------|--------------------------------------------|
| id_dossier_adoption | INT                                    | PK, AUTO_INCREMENT | Identifiant unique du dossier              |
| date_soumission     | DATE                                   | NON_NULL           | Date de dépôt de la demande                |
| statut_dossier      | ENUM('Validé', 'En Attente', 'Refusé') | NON_NULL           | État d'avancement du dossier               |
| commentaires_admin  | TEXT                                   | —                  | Observations de l'administrateur du refuge |
| id_animal           | INT                                    | FK, NON_NULL       | Animal concerné par la demande             |
| id_utilisateur      | INT                                    | FK, NON_NULL       | Utilisateur ayant soumis la demande        |

**SWIPER — Table d'association (swipe)**

Table de liaison représentant les swipes effectués par un utilisateur sur un animal.

| Champ          | Type | Contraintes      | Description                  |
|----------------|------|------------------|------------------------------|
| id_utilisateur | INT  | PK, FK, NON_NULL | Référence vers l'utilisateur |
| id_animal      | INT  | PK, FK, NON_NULL | Référence vers l'animal      |

**Index et optimisations**

| Table            | Index                       | Justification                                     |
|------------------|-----------------------------|---------------------------------------------------|
| Utilisateur      | (email)                     | Recherche et unicité par email                    |
| Animal           | (id_refuge)                 | Récupération rapide des animaux par refuge        |
| Animal           | (espece)                    | Filtrage performant par espèce                    |
| Dossier_Adoption | (id_utilisateur)            | Récupération des dossiers par utilisateur         |
| Dossier_Adoption | (id_animal)                 | Récupération des dossiers par animal              |
| SWIPER           | (id_utilisateur, id_animal) | Clé primaire composite — lookup rapide des swipes |

**6. Référence API**

Toutes les routes API utilisent `export const dynamic = 'force-dynamic'` pour désactiver le cache statique.

**6.1 Authentification Utilisateur**

`POST /api/register`

Inscription d'un nouvel adoptant.

| Paramètre | Type   | Requis | Description                                 |
|-----------|--------|--------|---------------------------------------------|
| email     | string | Oui    | Email de l'utilisateur                      |
| password  | string | Oui    | Mot de passe (haché avec bcrypt, 12 rounds) |
| firstName | string | Oui    | Prénom                                      |
| lastName  | string | Oui    | Nom                                         |

Réponses :

| Code | Description                                 |
|------|---------------------------------------------|
| 200  | <code>{ success: true, user: {...} }</code> |
| 400  | Email ou mot de passe manquant              |
| 409  | Email déjà utilisé                          |

`POST /api/login`

Connexion d'un utilisateur existant.

| Paramètre | Type   | Requis | Description  |
|-----------|--------|--------|--------------|
| email     | string | Oui    | Email        |
| password  | string | Oui    | Mot de passe |

Réponses :

| Code | Description                                 |
|------|---------------------------------------------|
| 200  | <code>{ success: true, user: {...} }</code> |
| 401  | Identifiants invalides                      |

## 6.2 Profil Utilisateur

`GET /api/profile?email={email}`

Récupère le profil d'un utilisateur.

`PUT /api/profile`

Met à jour le profil (nom, téléphone).

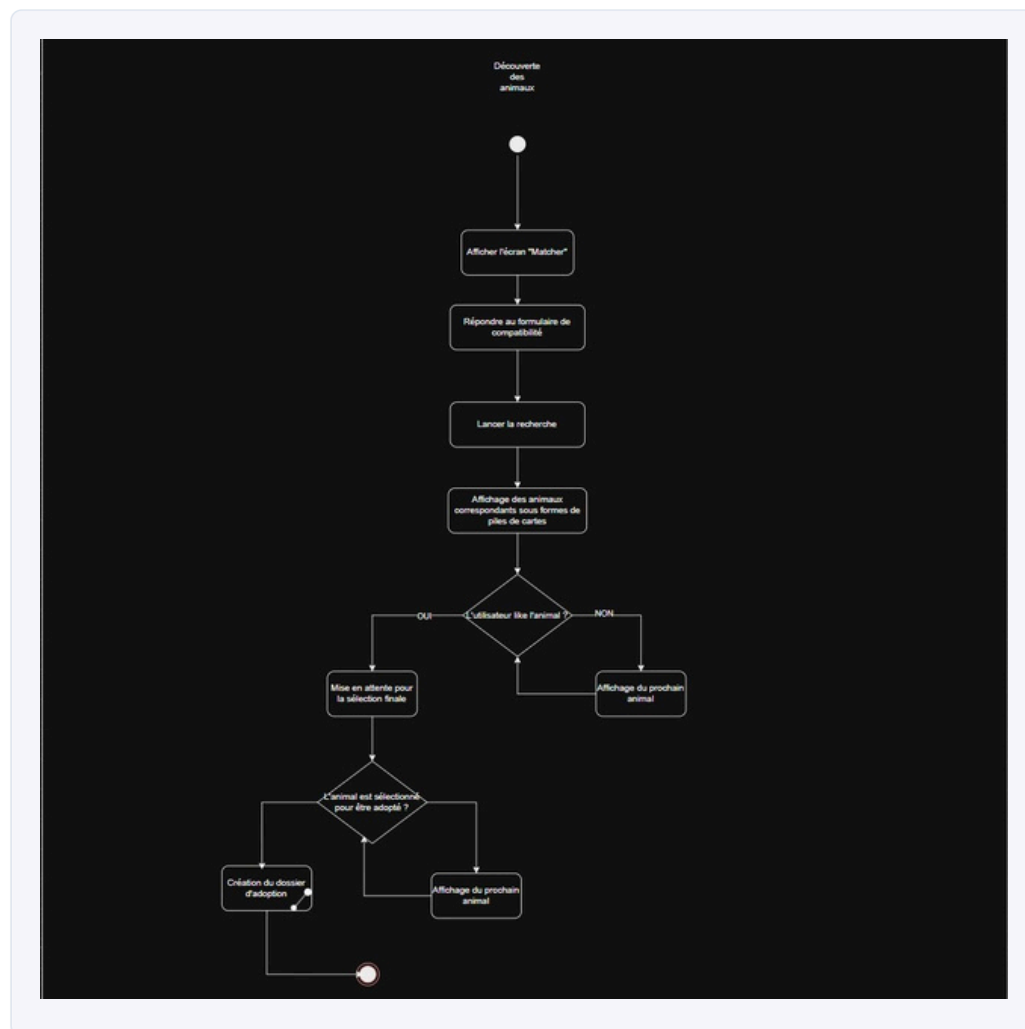
## 6.3 Matching & Recherche

`GET /api/match`

**Endpoint central** — Retourne les animaux compatibles avec le profil de l'adoptant.

| Paramètre (query)         | Type                 | Description                           |
|---------------------------|----------------------|---------------------------------------|
| <code>hasChildren</code>  | <code>boolean</code> | Foyer avec enfants                    |
| <code>hasOtherPets</code> | <code>boolean</code> | Autres animaux présents               |
| <code>hasGarden</code>    | <code>boolean</code> | Jardin disponible                     |
| <code>species</code>      | <code>string</code>  | Espèces recherchées (séparées par , ) |
| <code>age</code>          | <code>string</code>  | Catégories d'âge (séparées par , )    |





`GET /api/search`

Retourne tous les animaux disponibles sans filtre de compatibilité (limite : 50, triés par updatedAt DESC) Inclut la relation refuge.

## 6.4 Adoption

`POST /api/adopt`

Crée un dossier d'adoption. Upsert l'utilisateur si existant.

`GET /api/adoptions?email={email}`

Retourne les dossiers d'adoption d'un utilisateur.

## 6.5 Messagerie

`GET /api/messages?adoptionId={id}&viewerType={user|refuge}`

Récupère les messages d'un dossier et marque les messages comme lus.

`POST /api/messages`

Envoie un message et met à jour les drapeaux de lecture.

| Paramètre               | Type                | Requis | Description              |
|-------------------------|---------------------|--------|--------------------------|
| <code>adoptionId</code> | <code>int</code>    | Oui    | ID du dossier d'adoption |
| <code>senderType</code> | <code>string</code> | Oui    | "user" ou "refuge"       |
| <code>senderId</code>   | <code>int</code>    | Oui    | ID de l'émetteur         |
| <code>content</code>    | <code>string</code> | Oui    | Contenu du message       |

### 6.6 Administration (Refuges)

`POST /api/admin/register`

Inscription d'un nouveau refuge.

`POST /api/admin/login`

Connexion d'un refuge.

`GET /api/admin/animals?refugeId={id}`

Liste les animaux d'un refuge.

`POST /api/admin/animals`

Ajoute un animal au catalogue d'un refuge. Génère un `externalId` au format `MP-{refugeId}-{uuid8}`.

`GET /api/admin/adoptions?refugeId={id}`

Récupère les dossiers d'adoption liés à un refuge.

`PATCH /api/admin/adoptions`

Met à jour le statut d'un dossier d'adoption.

|                 |                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------|
| Statuts valides | <code>pending</code> · <code>reviewing</code> · <code>approved</code> · <code>rejected</code> |
|-----------------|-----------------------------------------------------------------------------------------------|

`DELETE /api/admin/adoptions?adoptionId={id}`

Supprime un dossier d'adoption.

## 7. Logique métier — Moteur de matching

### Principe

Le matching repose **exclusivement sur des critères de compatibilité** entre le profil de l'adoptant (renseigné via le questionnaire d'onboarding) et les contraintes de chaque animal.

### Critères de filtrage

| Critère adoptant                 | Contrainte animal             | Règle                                                               |
|----------------------------------|-------------------------------|---------------------------------------------------------------------|
| <code>hasChildren = true</code>  | <code>goodWithChildren</code> | Seuls les animaux compatibles avec les enfants sont proposés        |
| <code>hasOtherPets = true</code> | <code>goodWithDogs</code>     | Seuls les animaux compatibles avec les autres animaux sont proposés |
| <code>hasGarden = false</code>   | <code>needsGarden</code>      | Les animaux nécessitant un jardin sont exclus                       |
| Espèce(s) sélectionnée(s)        | <code>species</code>          | Mapping bilingue (ex : dog → Dog , Chien )                          |
| Catégorie(s) d'âge               | <code>age</code>              | Filtre par catégorie sauf si <code>any</code>                       |

### Principes d'exclusion

**Règle fondamentale :** un animal nécessitant un jardin ( `needsGarden = true` ) ne sera **jamais** proposé

à un utilisateur vivant en appartement ( `hasGarden = false` ).

- L'exclusion est **stricte** : un seul critère incompatible suffit à retirer l'animal des résultats.
- Le filtre `species` gère le **bilinguisme** (français/anglais) pour chaque espèce.
- Les résultats sont **limités à 50** et triés par date de mise à jour décroissante.

## 8. Expérience utilisateur

### Interface mobile-first

L'application est conçue prioritairement pour une utilisation sur smartphone avec :

- Navigation par **barre de navigation inférieure** ( `BottomNav` )
- Header responsive sur mobile
- Footer complet sur desktop

### Parcours utilisateur



### Interactions et animations

| Fonctionnalité         | Technologie              |                         | Description                                             |
|------------------------|--------------------------|-------------------------|---------------------------------------------------------|
| Système de swipe       | Framer                   | Motion                  | Navigation entre les profils d'animaux                  |
| Transitions de page    | Framer                   | Motion                  | Animations fluides entre les vues                       |
| Modèles 3D interactifs | React Three Fiber + Drei |                         | Rendu de modèles <code>.glb</code> (chien, chat, lapin) |
| Lazy loading 3D        | <code>React.lazy</code>  | + <code>Suspense</code> | Chargement différé des scènes 3D                        |

### Modèles 3D disponibles

| Fichier                          | Taille  | Utilisation                     |
|----------------------------------|---------|---------------------------------|
| <code>shiba_dog.glb</code>       | ~16 Mo  | Modèle principal page d'accueil |
| <code>an_animated_cat.glb</code> | ~3 Mo   | Chat animé                      |
| <code>rabbit.glb</code>          | ~6.7 Mo | Lapin                           |

### Images de fallback

Lorsqu'un animal n'a pas de photo, une image Unsplash est attribuée automatiquement en fonction de son espèce via la fonction utilitaire `getFallbackImage()` (supporte le français et l'anglais).

## 9. Installation & Développement

### Pré-requis

| Outil   | Version minimum      |
|---------|----------------------|
| Node.js | <code>22.22.2</code> |
| npm     | Inclus avec Node.js  |
| MySQL   | <code>9.4</code>     |

## Installation locale

```
# 1. Cloner le dépôt
git clone https://github.com/Kagitheepan/MatchPet.git
cd matchpet

# 2. Installer les dépendances
npm install

# 3. Configurer l'environnement
cp .env.example .env
# → Éditer .env avec vos identifiants de base de données
```

## Variables d'environnement

| Variable                  | Description               | Exemple                                                    |
|---------------------------|---------------------------|------------------------------------------------------------|
| <code>DATABASE_URL</code> | Chaîne de connexion MySQL | <code>mysql://user:password@localhost:3306/matchpet</code> |

## Initialisation de la base de données

```
# Générer le client Prisma
npx prisma generate

# Appliquer les migrations
npx prisma migrate dev

# (Optionnel) Charger les données de test
npx prisma db seed
```

## Lancer le serveur de développement

```
npm run dev
```

L'application sera accessible sur `http://localhost:3000`.

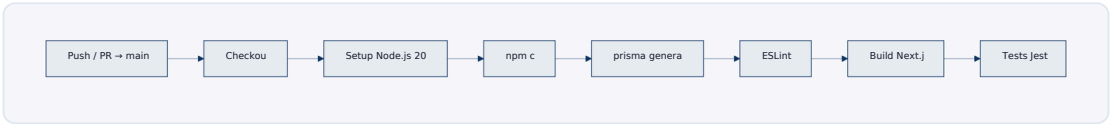
## Scripts disponibles

| Commande                   | Description                                                     |
|----------------------------|-----------------------------------------------------------------|
| <code>npm run dev</code>   | Serveur de développement Next.js                                |
| <code>npm run build</code> | Build de production                                             |
| <code>npm run start</code> | Démarrage production (avec <code>prisma migrate deploy</code> ) |
| <code>npm run lint</code>  | Vérification ESLint                                             |
| <code>npm test</code>      | Exécution des tests Jest                                        |

# 10. Déploiement & CI/CD

## Pipeline CI — GitHub Actions

Le pipeline s'exécute sur chaque `push` et `pull_request` vers la branche `main`.



Détails de la configuration :

| Étape   | Commande                             | Notes                                    |
|---------|--------------------------------------|------------------------------------------|
| Install | <code>npm ci</code>                  | Installation déterministe                |
| Prisma  | <code>npx prisma generate</code>     | Génération du client                     |
| Lint    | <code>npm run lint</code>            | ESLint avec config next                  |
| Build   | <code>npm run build</code>           | Utilise un DATABASE_URL factice          |
| Tests   | <code>npm test -- --runInBand</code> | Exécution séquentielle, 4 Go mémoire max |

## Production — Railway

| Élément         | Configuration                                                                |
|-----------------|------------------------------------------------------------------------------|
| Plateforme      | Railway                                                                      |
| Build           | <code>npm run build</code> (avec <code>postinstall: prisma generate</code> ) |
| Start           | <code>prisma migrate deploy ; next start -H 0.0.0.0 -p \${PORT}</code>       |
| Base de données | MySQL managée sur Railway                                                    |
| Variables       | Variables d'environnement sécurisées via Railway Dashboard                   |

**Portée actuelle :** L'application est utilisée dans le cadre du département de la **Seine-et-Marne (77)**.

## 11. Sécurité & Bonnes pratiques

### Mesures en place

| Mesure                    | Implémentation                                                                                  |
|---------------------------|-------------------------------------------------------------------------------------------------|
| Hachage des mots de passe | <code>bcryptjs</code> avec <code>12 salt rounds</code> - <code>bcrypt.hash(password, 12)</code> |
| Validation des entrées    | Vérification côté serveur dans chaque route API                                                 |
| Requêtes paramétrées      | Prisma ORM (protection contre les injections SQL)                                               |
| Singleton Prisma          | Instance unique en développement pour éviter les fuites de connexions                           |
| Force dynamic             | <code>export const dynamic = 'force-dynamic'</code> sur toutes les routes API                   |

### Configuration Prisma sécurisée

Le client Prisma utilise un pattern singleton avec support d'un **driver adapter MariaDB** :

```
// Tentative d'utilisation du driver adapter natif
const adapter = new PrismaMariaDb(connectionString)
client = new PrismaClient({ adapter })

// Fallback sans adapter si le driver n'est pas disponible
client = new PrismaClient()
```

### Recommandations futures

| Priorité | Mesure                                 | Description                                                |
|----------|----------------------------------------|------------------------------------------------------------|
| Haute    | <b>Authentification JWT / Sessions</b> | Remplacer le stockage côté client par des tokens sécurisés |
| Haute    | <b>Protection CSRF</b>                 | Tokens CSRF sur les formulaires                            |
| Moyenne  | <b>Rate limiting</b>                   | Limiter les appels API (protection brute force)            |
| Moyenne  | <b>CORS restreint</b>                  | Limiter les origines autorisées                            |
| Basse    | <b>Audit logging</b>                   | Journaliser les actions sensibles                          |

## 12. Maintenance

### Gestion de la base de données

**Règle absolue :** toute modification du schéma de base de données **doit** passer par Prisma.

```
# Créer et appliquer une migration
npx prisma migrate dev --name <nom_de_la_migration>

# Appliquer les migrations en production
npx prisma migrate deploy

# Réinitialiser la base (développement uniquement)
npx prisma migrate reset
```

### Bonnes pratiques

| Règle                                            | Détail                                                                              |
|--------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Ne jamais</b> modifier la base directement    | Toujours passer par <code>prisma migrate</code>                                     |
| <b>Toujours versionner les migrations</b>        | Le dossier <code>prisma/migrations/</code> doit être commité                        |
| <b>Toujours tester avant déploiement</b>         | Exécuter <code>npm run build</code> ; <code>npm test</code> localement              |
| <b>Mettre à jour le schéma Prisma en premier</b> | Le code applicatif découle du schéma                                                |
| <b>Seed reproductible</b>                        | Le fichier <code>prisma/seed.ts</code> doit produire des données de test cohérentes |

### Dépendances

```
# Vérifier les mises à jour disponibles
npm outdated

# Mettre à jour les dépendances (avec prudence)
npm update
```

## 13. Améliorations futures

| Priorité | Amélioration                       | Description                                                            |
|----------|------------------------------------|------------------------------------------------------------------------|
| Haute    | <b>Authentification robuste</b>    | JWT, sessions sécurisées, refresh tokens                               |
| Haute    | <b>Extension géographique</b>      | Étendre le périmètre au-delà de la Seine-et-Marne                      |
| Moyenne  | <b>Scoring de compatibilité</b>    | Score pondéré (0-100%) au lieu du filtrage binaire                     |
| Moyenne  | <b>Cache Redis</b>                 | Mise en cache des résultats de matching et des profils                 |
| Moyenne  | <b>Notifications en temps réel</b> | WebSocket / Server-Sent Events pour messages et statuts en temps réels |
| Moyenne  | <b>Géolocalisation active</b>      | Calcul de distance refuge - adoptant avec le <code>searchRadius</code> |
| Basse    | <b>PWA complète</b>                | Service Worker, mode hors-ligne, push notifications                    |
| Basse    | <b>Upload d'images</b>             | Stockage d'images animaux                                              |
| Basse    | <b>Tableau de bord analytics</b>   | Statistiques d'adoption pour les refuges                               |

**MatchPet** — Connecter les animaux aux familles qui leur correspondent.